

Coding Assignment 1

Due Date: *Monday, September 14 (11.59PM)*

Submission: on Gradescope

Instructions

The coding assignments may be completed in groups of *up to 4 students*. If you do so, please make sure that you include everyone's full name, and that you also *select everyone's name when submitting the assignment on Gradescope*. This ensures that each group member will get a grade assigned and have access to the comments from the graders.

For this assignment, the following items need to be submitted:

1. A **single PDF** that contains your code, all results, plots, and written answers (generated from R Markdown, Jupyter, etc.).
2. The **source file** used to produce the PDF (.Rmd, ipynb, ...).

Problem 1 [50 points]¹

In this coding assignment, we are going to use the **UCI Banknote Authentication Data Set**, which can be found and downloaded from the UCI Data Repository:

<https://doi.org/10.24432/C55P57>.

You can also read the details of the data collection process on the repository page.

Note: If you use Python, you can download the data directly from the UCI website or use pandas/sklearn to load it. If you use R, download the file `data_banknote_authentication.txt` from the repository above. You may also use any convenient built-in loader if available in your environment.

This data set contains **1,372 samples** of genuine and forged banknote-like specimens. Features were extracted from images using a *Wavelet Transform* tool, resulting in **4 continuous features**:

¹Logistic or other types of regression will not be given credit at this stage. The goal of this coding assignment is to get started with simpler tasks before diving into more sophisticated algorithms and methods.

1. variance of wavelet transformed image
2. skewness of wavelet transformed image
3. curtosis of wavelet transformed image
4. entropy of image

The response is a **binary class label** (0 = genuine, 1 = forged). There are no missing values.

Data preparation: Use a random 70% / 30% train–test split of the full data set. Set the random seed to 598 for reproducibility of the split.

Our **goal** is to compare the performance of a *simple linear regression* and a *k-nearest neighbor* classification approach on the data. Use a naive approach for the linear regression in which you treat the response as continuous and then convert the continuous predictions to a binary classifier by thresholding at 0.5. For the kNN approach, use $k = 1 : 20$.

- (a) [35 points] Plot the two errors (misclassification rate on the test set) against the corresponding k values. Make sure that you annotate the plots (e.g., using different colors or types of line) and add legends as needed. Also report the test error of the naive linear regression classifier as a horizontal reference line on the same plot.
- (b) [15 points] Does the plot match (approximately) your intuition of the bias–variance trade-off in terms of having a U-shaped error? What is the optimal k value based on this result? For the optimal k , what are the corresponding degrees-of-freedom and its error? Discuss.

Problem 2 [50 points]

The goal in this problem is to **write your very own kNN function!**

- (a) [5 points] Generate 3 independent standard Normal variables X_1, X_2, X_3 of $n = 1000$ independent observations. Generate the response Y from the following nonlinear model:

$$Y = X_1 + 2 \cdot X_2 - X_3 + 0.5 \cdot X_1 X_2 + \varepsilon$$

where the errors ε are IID $\mathcal{N}(0, 1)$. Set the random seed to 598 for reproducibility.

- (b) [10 points] Use the appropriate built-in kNN function in R or Python (Euclidean distance) and report the mean squared error (MSE) on the test set for $k = 1, 5, 10, 20$. Use the first 500 observations as the training data and the remaining 500 as the testing data.

- (c) *[35 points]* For this question, you **cannot** load any additional packages. Write your own kNN function

`mykNN(xtrain, ytrain, xtest, k)`

that fits a kNN model and predicts multiple target points `xtest`. The function should return `ytest`.

Notes:

- `xtrain` is the training data set feature matrix
- `ytrain` is the training data set response vector
- `xtest` is the testing data set feature matrix
- `ytest` is the testing data set vector of predictors

Test your implementation by computing the MSE on the testing data for the same values $k = 1, 5, 10, 20$. Your results should be very close to those obtained with the built-in function in part (b).

Recall that the MSE is defined as

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$